Tecnológico Nacional de México
Instituto Tecnológico de Tijuana

SUBDIRECCIÓN ACADÉMICA:

Departamento de Sistemas y Computación

SEMESTRE:

CARRERA: Ingeniería en informática

MATERIA: Patrones de diseño de software

Patrón State

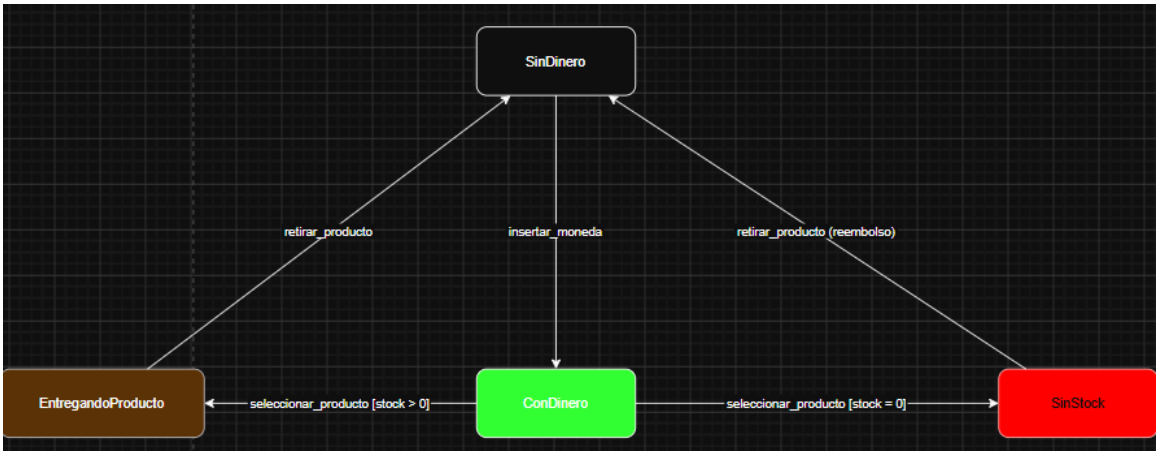
UNIDAD 3

Nombre: Alitzel Dalhi Vidales Hurtado No. Control: 21212371

DOCENTE: Maribel Guerrero Luis

Tijuana, Baja California, México Fecha: 28-04-2026

Diagrama de estados



Código:

```
import tkinter as tk
from abc import ABC, abstractmethod
import platform

def reproducir_sonido(tipo):
    if platform.system() == "Windows":
        import winsound
        try:
            if tipo == "moneda": winsound.Beep(1200, 150)
            elif tipo == "boton": winsound.Beep(800, 100)
            elif tipo == "error": winsound.Beep(300, 300)
            elif tipo == "entregar":
                winsound.Beep(600, 150)
                winsound.Beep(800, 200)
        except: pass
    else:
        print(f"🎵 [Efecto de sonido: {tipo}]")

# patrn state
class IState(ABC):
    @abstractmethod
    def insertar_moneda(self, context): pass

    @abstractmethod
    def seleccionar_producto(self, context, producto): pass

    @abstractmethod
    def retirar_producto(self, context): pass
```

```
    @abstractmethod
    def configurar_botones(self, btn_moneda, btn_retirar, botones_productos): pass

class SinDinero(IState):
    def insertar_moneda(self, context):
        reproducir_sonido("moneda")
        context.mensaje = "Moneda aceptada. Elige producto."
        context.state = ConDinero()

    def seleccionar_producto(self, context, producto): pass

    def retirar_producto(self, context): pass

    def configurar_botones(self, btn_moneda, btn_retirar, botones_productos):
        btn_moneda.config(state="normal", bg="#F1C40F")
        btn_retirar.config(state="disabled", bg="#7F8C8D", text="⬇️ Retirar")
        for btn in botones_productos: btn.config(state="disabled")

class ConDinero(IState):
    def insertar_moneda(self, context): pass

    def seleccionar_producto(self, context, producto):
        reproducir_sonido("boton")
        context.producto_seleccionado = producto
        if context.stock[producto] > 0: # <-- Este if evalúa inventario, no estado.
            context.mensaje = f"Preparando {producto}..."
            context.state = EntregandoProducto()
            reproducir_sonido("entregar")
        else:
            context.mensaje = f"ERROR: {producto} agotado."
            context.state = SinStock()
            reproducir_sonido("error")

    def retirar_producto(self, context): pass

    def configurar_botones(self, btn_moneda, btn_retirar, botones_productos):
        btn_moneda.config(state="disabled", bg="#7F8C8D")
        btn_retirar.config(state="disabled", bg="#7F8C8D", text="⬇️ Retirar")
        for btn in botones_productos: btn.config(state="normal")

class EntregandoProducto(IState):
    def insertar_moneda(self, context): pass
    def seleccionar_producto(self, context, producto): pass

    def retirar_producto(self, context):
```

```
        reproducir_sonido("boton")
        prod = context.producto_seleccionado
        context.mensaje = "MÁQUINA LISTA. Inserta moneda."
        context.stock[prod] -= 1
        context.producto_seleccionado = None
        context.state = SinDinero()

    def configurar_botones(self, btn_moneda, btn_retirar, botones_productos):
        btn_moneda.config(state="disabled", bg="#7F8C8D")
        btn_retirar.config(state="normal", bg="#2ECC71", text="⬇️ Retirar de la bandeja")
        for btn in botones_productos: btn.config(state="disabled")

class SinStock(IState):
    def insertar_moneda(self, context): pass
    def seleccionar_producto(self, context, producto): pass

    def retirar_producto(self, context):
        reproducir_sonido("boton")
        context.mensaje = "MÁQUINA LISTA. Inserta moneda."
        context.producto_seleccionado = None
        context.state = SinDinero()

    def configurar_botones(self, btn_moneda, btn_retirar, botones_productos):
        btn_moneda.config(state="disabled", bg="#7F8C8D")
        btn_retirar.config(state="normal", text="Reembolsar y Reset", bg="#E74C3C")
        for btn in botones_productos: btn.config(state="disabled")

class Context:
    def __init__(self, state):
        self.state = state
        self.stock = {"🥤 Coca Cola": 5, "🍟 Papas": 3, "🍫 Chocolate": 2}
        self.producto_seleccionado = None
        self.mensaje = "MÁQUINA LISTA. Inserta moneda."

    def insertar_moneda(self): self.state.insertar_moneda(self)
    def seleccionar_producto(self, producto): self.state.seleccionar_producto(self, producto)
    def retirar_producto(self): self.state.retirar_producto(self)
    def configurar_botones(self, btn_moneda, btn_retirar, botones_productos):
        self.state.configurar_botones(btn_moneda, btn_retirar, botones_productos)

# ui
context = Context(SinDinero())

def actualizar_ui():
    estado_actual = type(context.state).__name__
```

```
    pantalla_estado.config(text=f"EST: {estado_actual}")
    pantalla_mensaje.config(text=f"> {context.mensaje}")

    texto_vitrina = "\n"
    for prod, cant in context.stock.items():
        disp = "🟢 Disponible" if cant > 0 else "🔴 Agotado"
        texto_vitrina += f"{prod}\n{disp} ({cant} unds)\n\n"
    vitrina_label.config(text=texto_vitrina)

    # El estado actual decide cómo configurar los botones
    context.configurar_botones(btn_moneda, btn_retirar, botones_productos)

def insertar_moneda():
    context.insertar_moneda()
    actualizar_ui()

def seleccionar_producto(nombre_producto):
    context.seleccionar_producto(nombre_producto)
    actualizar_ui()

def retirar_producto():
    context.retirar_producto()
    actualizar_ui()

ventana = tk.Tk()
ventana.title("Máquina Expendedora")
ventana.geometry("550x450")
ventana.configure(bg="#1E1E1E")
ventana.resizable(False, False)

frame_izquierdo = tk.Frame(ventana, bg="#2C3E50", bd=8, relief="groove")
frame_izquierdo.pack(side="left", fill="both", expand=True, padx=10, pady=10)

frame_derecho = tk.Frame(ventana, bg="#34495E", bd=8, relief="raised")
frame_derecho.pack(side="right", fill="y", padx=10, pady=10)

tk.Label(frame_izquierdo, text="CRISTAL", font=("Arial", 8), bg="#2C3E50",
fg="#5D6D7E").pack(pady=5)
cristal = tk.Frame(frame_izquierdo, bg="#111111", bd=5, relief="sunken")
cristal.pack(fill="both", expand=True, padx=10, pady=(0, 10))

vitrina_label = tk.Label(cristal, text="", font=("Helvetica", 11, "bold"), bg="#111111",
fg="#ECF0F1", justify="center")
vitrina_label.pack(expand=True)

bandeja = tk.Frame(frame_izquierdo, bg="#000000", height=60, bd=5, relief="sunken")
```

```
bandeja.pack(fill="x", padx=15, pady=10)
tk.Label(bandeja, text="P U S H", font=("Arial", 12, "bold"), bg="#000000",
fg="#4d4d4d").pack(pady=15)

# Pantalla
pantalla = tk.Frame(frame_derecho, bg="#0A0A0A", bd=4, relief="sunken")
pantalla.pack(pady=10, padx=10, fill="x")

pantalla_estado = tk.Label(pantalla, text="...", font=("Courier", 10, "bold"), bg="#0A0A0A",
fg="#00FF00", anchor="w")
pantalla_estado.pack(fill="x", padx=5, pady=5)
tk.Frame(pantalla, bg="#005500", height=1).pack(fill="x", padx=5)
pantalla_mensaje = tk.Label(pantalla, text="...", font=("Courier", 10, "bold"), bg="#0A0A0A",
fg="#00FFFF", anchor="w", wraplength=170)
pantalla_mensaje.pack(fill="x", padx=5, pady=5)

# Ranura
tk.Label(frame_derecho, text="🪙 Ranura", font=("Arial", 9, "bold"), bg="#34495E",
fg="white").pack(pady=(10, 0))
btn_moneda = tk.Button(frame_derecho, text="Insertar Moneda", font=("Arial", 10, "bold"),
cursor="hand2", command=insertar_moneda)
btn_moneda.pack(pady=5, fill="x", padx=15)

# Teclado Numérico
tk.Label(frame_derecho, text="🔢 Teclado", font=("Arial", 9, "bold"), bg="#34495E",
fg="white").pack(pady=(10, 0))
botones_productos = []
for prod in context.stock.keys():
    nombre_limpio = prod.split(" ")[1]
    btn = tk.Button(frame_derecho, text=f"Elegir {nombre_limpio}", font=("Arial", 10),
                    bg="#ECF0F1", cursor="hand2",
                    command=lambda p=prod: seleccionar_producto(p))
    btn.pack(pady=3, fill="x", padx=15)
    botones_productos.append(btn)

# Bandeja de Retiro
tk.Frame(frame_derecho, bg="#7F8C8D", height=2).pack(fill="x", padx=10, pady=(20, 10))
btn_retirar = tk.Button(frame_derecho, text="⬇️ Retirar", font=("Arial", 10, "bold"),
cursor="hand2", command=retirar_producto)
btn_retirar.pack(pady=5, fill="x", padx=10)

actualizar_ui()
ventana.mainloop()
```

Conclusión

Este patrón es una alternativa bastante útil para evitar llenar nuestro código de if else, aparte es bastante sencillo de usar, a comparación de otros códigos que he realizado, este en solo doscientas líneas de código ya está el programa completo, también hace ver nuestro código más limpio y ordenado. Con relativamente poco código logramos un código extensible y fácil de entender para cualquier desarrollador que tenga que trabajar con él después.